

Formal Verification of Fault-Tolerant Token-Based Cache Coherence for Chip Multiprocessors

Advait Iyer Arjun Laxman Mustafa Miyaziwala
advayer@umich.edu arlx@umich.edu mmiyazi@umich.edu

Noel Pamenan Sanidhya Patel
pamenann@umich.edu sanmapat@umich.edu

Abstract—Token-based cache coherence protocols use unordered interconnects to broadcast messages, avoiding indirection latency while maintaining a high bandwidth. FtTokenCMP, a fault-tolerant token-based protocol, purportedly ensures correctness even in a chip multiprocessor (CMP) system with a fault-prone interconnect. This claim is analyzed using the FtTokenCMP’s authors’ underlying assumptions about processor and network behavior. Shortcomings in the FtTokenCMP protocol concerning coherence in a fault-prone interconnect and a possible solution are presented. Additionally, formal verification attempts with Murphi are discussed.

1. Introduction

Transistors have shrunk in size following Moore’s Law[1] which has allowed manufacturers to pack more and more transistors onto a single chip. This has led to the introduction of Chip Multiprocessors(CMPs)[2] which have processors, caches, memory controllers, etc. all integrated on a single die. They allow for a significant increase in performance without too much power consumption and can exploit thread-level parallelism which is supplemented by a high-bandwidth, low-latency interconnect that does not use any glue logic such as switches.

These multiprocessors each have their caches and thus introduce the need for a cache coherence protocol. Bus-based snooping protocols were introduced to work with a virtual bus which is an interconnect with multiple switches and a root switch. Their main goal was to reduce cache-to-cache miss latencies. However, they use a broadcast system as a result of which they have poor scalability[8] and are only designed to work with relatively smaller multiprocessors with around 4-16 individual processors. Moreover, they do not work with the glueless interconnect mentioned previously due to the lack of a virtual bus system. For this reason, directory protocols are used. They do not require broadcasts or an ordered interconnect. Instead, they rely on a home directory, usually in main memory, which responds to requests sent by the processors or forwards them to the appropriate processor. Since the messages sent on the interconnect are targeted, the bandwidth consumption is reduced but miss latency is increased[3].

To combat the shortcomings of these cache coherence protocols, a third type of protocol called token-based cache coherence protocol is introduced which has low-latency cache-to-cache misses, does not rely on a bus-based interconnect, and has good bandwidth efficiency.

2. Background

2.1. Basic Token Coherence

Token coherence protocol works on the base concept of MOESI but implements it using “tokens” which are like units of permission that allow the processor to perform certain actions on a cache block depending upon the number of tokens it possesses. This protocol has two parts: a correctness substrate and a performance policy.

2.1.1. Correctness Substrate. The main goal of this substrate is to implement the correct protocol on the unordered interconnect. It does this by ensuring safety and avoiding starvation, by allowing a processor to read or write if it has the right number, and type, of tokens. It needs a minimum of one token to read, and all the tokens in the system(equivalent to the number of processors) to write to a cache block. Starvation avoidance is implemented using persistent requests. These are special requests that are issued when a processor is starving for tokens. Persistent requests have to be processed and the requestor eventually has to get the data and/or tokens it has requested regardless of any other conflicting requests in the system. These requests are bandwidth-heavy and are requested infrequently[4].

2.1.2. Performance Policies. As a result of the correctness substrate, we can implement several performance policies which are done through transient requests. They are meant to be fast and bandwidth efficient without worrying about correctness as that is now the responsibility of the correctness substrate. These are lighter requests issued by processors requesting tokens and/or data but are not guaranteed to succeed due to concurrent requests or protocol races. However, most of these requests succeed in the common case which contributes to the bandwidth efficiency of token coherence protocols[4].

2.2. Source of Faults

Due to the decreasing size of transistors and higher frequencies, CMPs are more susceptible to transient faults. These are single-event errors that cause a component to produce one incorrect output and continue to function normally. These failures can be caused by alpha-particle strikes, cosmic rays, and radiation from radioactive atoms which tend to exist in all materials and especially electrical sources[5]. Such faults can occur in the interconnect network of CMPs. These may lead to messages being deleted from the system or arriving at their destination corrupted or at the wrong destination altogether, both of which are eventually discarded. While some other solutions using techniques such as checksums[6] help mitigate this, it can be useful to tackle it at the coherence level. This has led to the modification of basic token coherence protocols to token-based fault-tolerant cache coherence protocols[10][11]. We attempted to formally verify one such protocol called FtTokenCMP using Murphi [7].

3. Description of Design

FtTokenCMP is only designed to maintain coherence if coherence messages have two possibilities: correctly arriving at their destination or not arriving at all. Error detection codes are used to guarantee that no incorrect or corrupted messages can be processed by a node[10].

While losing a transient request is harmless (since it is a component of the performance policy, independent of correctness), the following situations need to be mitigated:

- Losing a persistent request: results in deadlock since a persistent request cannot be reissued.
- Losing a message carrying tokens: eventually results in deadlock as the total number of tokens in the system decreases. The effect of this decrease is that no processor will be able to write to that cache line.
- Losing a message carrying the owner token: not only results in deadlock, but it could also result in data loss if the owner token is dirty.

3.1. States Added By FtTokenCMP

FtTokenCMP adds more states to the traditional MOESI states used in TokenCMP.

Backup (B): This state is entered when a processor has ownership permissions (M, O, or E) and it receives an external request that requires sending out the ownership token. In this state, the cache will keep a ‘backup’ copy of the data that it sent out in case that data is needed for recovery for a token recreation process. This state is analogous to invalid so it cannot service normal transient requests for data. A line will transition from backup to invalid once an ownership acknowledgement or backup invalidate message is received.

Blocked ownership (Mb, Eb, and Ob): To prevent having multiple backup values for a line at any given time, a cache that receives the owner token cannot respond to external requests that would require sending out the owner token until it leaves its ‘blocked’ ownership state. It can only leave this blocked state once it receives a backup deletion acknowledgement or set token serial number message.

Recreating tokens (R): This state is entered when a fault is detected and a token recreation process is initiated [11]

3.2. Timeouts as Fault Recovery Mechanisms

Timeout	When is it activated?	Where is it activated	Where is it deactivated?	What happens when it triggers?
Lost Token	When a persistent request becomes active.	At the starver cache.	When the persistent request is satisfied or deactivated.	Request a token recreation.
Lost Data	When a backup state is entered. (When the owner token is sent.)	At the cache that holds the backup.	When the backup state is abandoned. (When the Ownership Acknowledgement Arrives.)	Request a token recreation.
Lost Backup Deletion Acknowledgement	When a line in a blocked status needs to be replaced.	At the cache that needs to do the replacement.	When the blocked state is abandoned. (When the Backup Deletion Acknowledgement arrives.)	Request a token recreation.
Lost Persistent Deactivation	When a persistent request from another cache is activated.	At every cache (by the persistent request table).	When the persistent request is deactivated.	Send a persistent request ping.

TABLE 1. DESCRIPTION OF VARIOUS TIMEOUTS IN A TOKEN-BASED CACHE COHERENCE PROTOCOL[10]

FtTokenCMP uses timeouts as a corrective mechanism for transactions that could have potentially been faulted. This is implemented through a timeout mechanism which activates persistent and token recreation requests that prevent:

- **Token Loss:** If a processor tries to write to a memory line with a lost token, it will be unable to acquire all the tokens required to enter a Modified state, even after a persistent request. This would eventually trigger a token recreation request, resetting the state of the cache line.

- **Data Loss:** In order to ensure data transmitted across the network does not get lost due to a fault, any sender with dirty data is required to store a ‘backup’ (that cannot be evicted or service internal requests) before sending it. This line cannot be evicted until an ownership acknowledgement is received as confirmation of successful data delivery. A lost data timeout is associated with waiting for this acknowledgement, triggering a token recreation request as a safeguard when exceeded. There is a corresponding lost backup deletion timeout as well, which is set off when the deletion acknowledgement does not reach the new owner in time[10].
- **Persistent Request Activation Loss:** Persistent requests help prevent starvation, and are recorded using a distributed-arbitration persistent request scheme. This opens up possibilities that persistent requests may never be activated, may get lost, and may not be deactivated in time. A lost token time-out will be triggered by the persistent requester, resulting in token recreation.
- **Persistent Request Deactivation Loss:** If a persistent request deactivation message is lost, a processor may be left with a persistent request that remains indefinitely active. To address this, a lost persistent deactivation time-out is initiated when a persistent request from another cache is activated; this time-out triggers the cache to send a persistent request ping to the requester. Upon receiving this ping, the requester will reply with either a persistent request or a deactivation message based on the current status of the request for that line, with the time-out resetting after each ping is sent[11].

3.3. Token Recreation

Token recreation is the primary fault recovery mechanism provided by FtTokenCMP[11]. This process is initiated whenever the system detects a loss of a coherence message via the timeouts described previously. By the end of this process, there will be one, and only one copy of the data with all the tokens that traditionally exist in the system. This is guaranteed to happen except when no valid copy of the data exists, only the backup copy exists, and the token recreation process was requested by a cache that does not have the backup copy of the data. However, at least according to their protocol, it is guaranteed that a backup copy of the data still exists in one of the caches so it is guaranteed that the lost data time-out will eventually trigger and tokens will be recreated which prevents deadlock.

To ensure that there are no exist more tokens than processors in the system, token validity is determined using an associated serial number—with the latest serial number indicating it is valid. Every time a token recreation request is initiated (due to a timeout), the memory controller increments the serial number, enters a recreating state, and broadcasts this serial number (through a set token serial message, TrS).

Caches send back TrSack messages, along with valid data if applicable. The one holding the backup token retains its state. For verification purposes, we only sent back data from the owner, in order to mitigate network congestion, a performance policy difference.

When memory controller receives a TrSack with data, it broadcasts a backup invalidate (BInv) message, which invites a backup invalidation acknowledgement (BInvAck) from the backup token holder. Once the memory controller has received all the expected acknowledgements, it broadcasts a destruction done (TrDone) message to the initiator (if not memory), with data if applicable.

When a cache receives TrDone with data, it will recreate all the tokens (with the new token serial number). If a cache receives a TrDone message without data it can only recreate the tokens if it has the backup data (in backup state).

3.4. FtTokenCMP Invariants

The protocol requires the following invariants[7] to be upheld. These were used to design our formal verification program in Murphi.

- **Conservation of Tokens:** Each line of shared memory has a fixed number of $T + 1$ tokens associated with it. Once the system is initialized, tokens may not be created or destroyed. One token for each block is the owner token (clean or dirty), and a backup token.
- **Write Rule:** A component can write a block only if it holds all T tokens for that block which are not the backup tokens and has valid data. After writing the block, the owner token is set to dirty.
- **Read Rule:** A component can read a block only if it holds at least one token different than the backup token for that block and has valid data.
- **Data Transfer Rule:** If a coherence message carries a dirty owner token, it must contain data.
- **Owner Token Transfer Rule:** If a coherence message carries the owner token, it must not carry the backup token as well.
- **Backup Token Transfer Rule:** The backup token can only be sent to another node that already holds the owner token.
- **Blocked Ownership Rule:** The owner token cannot be sent to other component until the backup token has been received.
- **Valid-Data Bit Rule:** A component sets its valid data bit for a block when a message arrives with data and at least one token different than the backup token. A component clears the valid-data bit when it no longer holds any tokens or when it holds only the backup token. The home memory sets the valid-data bit whenever it receives a clean owner token, even if the message does not contain data.
- **Clean Rule:** Whenever the memory receives the owner token, the memory sets the owner token to clean.

4. Methodology

4.1. Murphi Model Checker

Murphi[7] is an explicit state model checker mainly used for formal verification of cache coherence protocols. Murphi uses a model-checking approach where the described protocol is modeled as a finite state machine[9]. Murphi uses user-defined rules to generate all possible states and explores each transition to ensure the correctness of the protocol including safety and liveness, while also checking for user-defined invariants at each state.

As systems increase in complexity, the number of states that Murphi has to check increases exponentially and becomes computationally infeasible. To combat this, Murphi employs a technique called symmetry reduction. In a system where there are multiple identical processors, the identity of the processors becomes irrelevant to the verification process. Murphi leverages this symmetry and combines several transitions that are functionally equivalent into a single transition and tests this for a single permutation of processors. A Murphi programmer uses multisets and scalarsets to encode symmetry.

Our primary reasoning behind using Murphi was its ease of use, familiarity with the software, and its ability to prove the absence of deadlocks, livelocks and various race conditions which was paramount to ensuring a complete implementation of the described protocol.

4.2. Modeling Coherence Message Delivery

TokenCMP cache coherence, the base cache coherence protocol upon which fault tolerance is added, can exploit an unordered interconnect while avoiding indirection by broadcasting all coherence messages.

To model this unordered interconnect while leveraging symmetry through multisets, an array of multisets of coherence messages was used. Each node owns one of the multisets within the array, where a node refers to a processor or main memory. When a node wants to send a coherence message, it broadcasts by populating each multiset within the array, excluding its own, with a copy of the message.

When a node's message multiset is non-empty, two rules are eligible to fire:

- 1) An arbitrary message is chosen from the multiset. If the message's destination node field is either undefined (used for broadcasts) or matches the receiving node, the message is used to update the state of the node. Since different nodes can choose messages from their multisets in different orders, an unordered interconnect is effectively modeled.
- 2) An arbitrary message is removed from the multiset without updating the state of the node. We call this a fault injection.

Since Murphi is an exhaustive model checker, bounds and safeguards need to be added on top of the two basic rules to remove cases that would not exist in real systems.

4.2.1. Token Serial Number Aliasing. Token serial numbers allow processors to disregard outdated coherence messages that were in flight before a token recreation process. The size of the token serial number should be set such that when it wraps around and a serial number is reused by the system, there cannot be an in-flight message with the same serial number. In real systems, the maximum required size of the token serial number is bounded; this size would be a function of the maximum time a message can be in the interconnect divided by the time required for token recreation.

However, since the message multisets used to model the unordered interconnect in Murphi allow messages to remain in flight for an arbitrarily long period, there is no set bound for the maximum token serial number size. To mitigate this, whenever main memory increments the token serial number, all in-flight messages with the same serial number as the new one are removed from all message multisets. This is safe to do since serial number aliasing is not possible in real systems given a large enough serial number.

4.3. Processor Model

Each processor record carries the state it wants the cache line: invalid (no tokens), shared (at least one token), or modified (all the tokens). A processor can change its desired state, through a series of Murphi rules, only once it has met its current desired state. For modeling, all processors start in an invalid state with their desired state being invalid.

4.3.1. Transient Requests. To reach its desired state, processors start by making transient requests. These transient requests are either GetS (to request the cache block in the shared state), or GetM (to request the cache block in the modified state). A processor can re-broadcast transient requests a bounded number of times (three in our model) before it is no longer eligible to issue transient requests and transitions to issuing a persistent request. Our model differs from FtTokenCMP concerning transient requests only in the lack of a strict timeout period before a processor is eligible to issue another transient request. However, we found it unnecessary to model this timeout for formal verification.

4.3.2. Initiating Persistent Requests. Once a processor has broadcasted the requisite number of transient requests, it can broadcast a persistent request. Section 4.5 discusses how nodes handle the reception of persistent requests.

After a processor has recorded in its state that it has issued a persistent request, it is eligible for the set of timeouts associated with the token recreation process. A processor can issue a maximum of one persistent request to achieve a desired state.

4.4. Main Memory Model

The main memory node starts by holding all tokens and sends them out to processors upon receiving transient

or persistent requests. Cache evictions in processor nodes results in main memory receiving tokens.

For FtTokenCMP, the main memory node is responsible for arbitrating the token recreation process. The following are the relevant elements of the main memory node's record in Murphi associated with token recreation:

- Token Recreation Requester: the source node of the current token recreation request. Per FtTokenCMP, an active token recreation request cannot be superseded with another recreation request.
- Token Invalidation Acknowledgment Count: keeps track of the number of token invalidation acknowledgments received from processor nodes. Recreation can progress to backup invalidation once all acknowledgments are received.
- Backup Invalidation Acknowledgement Count: keeps track of the number of backup invalidation acknowledgments received from processor nodes. Tokens can be reconstructed safely once all acknowledgments are received.

Main memory is eligible to trigger a token recreation request (through the token recreation timeout) given that the token recreation requester is defined.

4.4.1. Bounding Token Recreation. In Murphi's state search, messages associated with token recreation can continuously be faulted, halting forward progress and pushing the number of in-flight messages to the maximum bound. For example, before adding bounds, the model failed a "too many network messages" assertion after faulting a backup invalidation acknowledgment five times in a row.

Bounds ensure that forward progress is eventually made. These bounds are necessary given that it is unreasonable to design a cache coherence protocol whose coherence messages can be faulted an unbounded number of times.

For token recreation, two complementary global structures are used to implement bounding:

- 1) Timeout Count Table: an array that stores the number of times each processor has triggered a timeout on a token recreation request. Once this count for a processor reaches a maximum bound, the processor is ineligible to start a new token recreation request, and associated messages associated with its final request are protected from faulting. The count is reset upon successful completion of token recreation.
- 2) Fault Count Table: an array that, for each processor, stores the number of messages within a token recreation transaction that has been faulted. Once this count reaches a maximum bound, messages associated with the transaction are protected from faulting. This structure enables control over faults at the transaction level.

4.5. Handling Persistent Requests

Upon receiving a persistent request, a node updates a local table that keeps track of all active persistent requests

(including its own in the case of processor nodes). In Murphi, this table is an array of booleans, with an element for each processor. A boolean state of true at the i th element of the array indicates that the node has received a persistent request from the i th processor.

Amongst all the active persistent requesters, a node chooses the requester with the smallest ID to send or forward future received tokens to. This policy prevents a livelock condition in which processors indefinitely ping-pong tokens back and forth, each thinking the other is the persistent requester. When a deactivate persistent request message is received (or when a processor has satisfied its desired state), the appropriate field of the persistent request table is set to false, and the next persistent requester, if any, is found with the smallest ID policy.

4.5.1. Bounding Deactivation Timeouts. FtTokenCMP includes a persistent request deactivation timeout that protects a node from being stuck in a persistent requestee state given a faulted deactivation message. A deactivation timeout triggers the node to send a ping to the node it thinks is the current persistent requester to check whether that is still the case. In Murphi, a node is eligible to send a ping as long as it is a persistent requestee.

For the same reasons discussed in section 4.4.1, deactivation timeouts need to be bounded. To accomplish this, each node keeps track of the number of times it has sent a persistent request ping. When this number reaches a maximum bound, the messages associated with the transaction are protected from faulting.

5. Results

FtTokenCMP is not a valid protocol as currently constructed. More specifically, it is possible to lose data which can cause incoherence or deadlock.

5.1. Deadlock on Fault of TrS Acknowledgment

Say there are two processors in the system and processor 1's cache (C1) is in state M and processor 2's cache (C2) is in state I. The following sequence is possible:

- 1) C2 broadcasts a GetM
- 2) C1 receives the GetM request and sends out all of its tokens and data to C2 and enters state B
- 3) C2 receives the tokens and data from C1, enters state Mb, and sends AckO to C1
- 4) C1 receives AckO and deletes its backup copy causing it to transition to state I. It then sends AckBD to C2
- 5) The AckBD message experiences a transient fault and gets deleted from the system
- 6) Lost Backup Deletion Time-out triggers which causes C2 to issue a TrR to the memory controller
- 7) The memory controller receives the TrR, increments token serial number, and sends out TrS

- 8) C1 receives the TrS and sends out a TrSAck to memory
- 9) C2 receives the TrS, destroys all tokens causing it to transition to invalid, and sends out a TrSAck+Data to memory
- 10) The TrSAck+Data message experiences a transient fault and gets deleted from the system
- 11) Token Recreation Time-out is triggered so memory increments its token serial number and sends out TrS

At this point, neither processor has valid data and memory also does not have valid data. No forward progress can be made as no tokens are allowed to be recreated. This is, for all intents and purposes, deadlock.

5.2. Incoherence from Internal Stores in Mb

In the protocol, it is implied that a cache in state Mb is able to service internal store requests. The only restriction that explicitly exists is that the cache cannot transfer ownership to a different cache while in this state, "the cache will ignore external requests to write to that line". Again, let's say there are two processors in the system and processor 1's cache (C1) is in state M and processor 2's cache (C2) is in state I but this time C1 has data value 1. The following sequence is possible:

- 1) C2 broadcasts a GetM
- 2) C1 receives the GetM request and sends out all of its tokens and data to C2 and enters state B
- 3) C2 receives the tokens and data from C1, enters state Mb, and sends AckO to C1
- 4) C2 stores value 2
- 5) The AckO message experiences a transient fault and gets deleted from the system
- 6) Lost Data Time-out triggers which causes C1 to issue a TrR to the memory controller
- 7) The memory controller receives the TrR, increments token serial number, and sends out TrS
- 8) C1 receives the TrS and sends out a TrSAck to memory
- 9) C2 receives the TrS, destroys all tokens causing it to transition to invalid, and sends out a TrSAck+Data to memory
- 10) The TrSAck+Data message experiences a transient fault and gets deleted from the system
- 11) Token Recreation Time-out is triggered so memory increments its token serial number and sends out TrS

At this point, C1 is still in state B while C2 is in state I. The backup token does exist so eventually tokens will be recreated. However, the last write to the system got lost when the TrSAck+Data message got deleted so memory will set its value to 1 instead of 2 which is incoherent.

5.3. Potential Solution

A potential solution to both of these problems is having the cache not transition to invalid if it is in a blocked

ownership state and it receives a TrS message. This would ensure that the potential only valid copy of data cannot be lost. Instead, a cache in a blocked ownership state will simply eliminate all of its sharer tokens if it receives a TrS message but it will keep the owner token. This ensures that the cache cannot service any further internal store requests as not enough tokens exist in the system to service another store request. Essentially, the TrSAck+Data message is guaranteed to carry the most up-to-date value in the system.

Now, if the TrSAck+Data message never arrives we still have a valid copy of the data in at least one cache. If the message does arrive, a BInv message will get sent out which will clear any remaining backup token and/or owner token in the system before tokens are recreated.

To implement this solution in Murphi, we modified how caches responds to TrS messages in our switch statements and added an element in the memory controller's record called 'OwnerAck' which acts as a valid bit for the data in memory. Normally valid bits are not necessary as tokens serve as a 'validation' check, but actually gaining the owner token due to TrSAck+Data can cause there to be multiple owners in the system which violates one of the original protocol's invariants.

5.4. Problems with Murphi Verification

Even with our additions to FtTokenCMP, we were unable to formally verify the protocol. This failure was not a result of the incorrectness of the protocol but of our implementation of the protocol in Murphi.

Even after adding bounds, limiting the maximum number of in-flight messages, eliminating performance messages, and reducing the number of nodes, the state space explodes beyond memory constraints. We also played with Murphi parameters like "gPercentActiveStates" which increases the search queue size and execution flags that change the search behavior, but still were not able to get the model to finish validation. Currently, Murphi explores about five million states without error before terminating on memory constraints. We also attempted to set the start state such that Murphi more quickly reaches the state in which incoherence should occur, but this approach was also not fruitful.

Given more time, we would attempt to exploit more symmetry to reduce the state space that Murphi has to search.

6. Conclusion

In this paper, we have shown that there are problems with incoherence in FtTokenCMP. In particular, deadlock or data loss is possible due to a fault of a set token serial number acknowledgment. We did not find any problems that fundamentally broke the basis of the protocol so we attempted to formally verify FtTokenCMP, with our proposed fixes, in Murphi. Due to time constraints, we were unable to fully implement the protocol in Murphi. As future work, we would like to complete our implementation in Murphi

to prove whether FtTokenCMP is fundamentally flawed or if it is a protocol that can work with appropriate fixes.

7. Acknowledgments

We would like to thank our EECS 570 instructors: Professor Ronald Dreslinski, Jonah Rosenblum, and Joy Dong.

References

- [1] G. E. Moore, "Cramming more components onto integrated circuits, Reprinted from Electronics, volume 38, number 8, April 19, 1965, pp.114 ff.," in IEEE Solid-State Circuits Society Newsletter, vol. 11, no. 3, pp. 33-35, Sept. 2006, doi: 10.1109/N-SSC.2006.4785860.
- [2] Luiz André Barroso, Kourosh Gharachorloo, Robert McNamara, Andreas Nowatzky, Shaz Qadeer, Barton Sano, Scott Smith, Robert Stets, and Ben Verghese. 2000. Piranha: a scalable architecture based on single-chip multiprocessing. SIGARCH Comput. Archit. News 28, 2 (May 2000), 282–293. <https://doi.org/10.1145/342001.339696>.
- [3] A. D. Joshi, S. Indrajeet, N. Ramasubramanian and B. S. Begum, "Analysis of multi-core cache coherence protocols from energy and performance perspective," 2017 International Conference on Recent Innovations in Signal processing and Embedded Systems (RISE), Bhopal, India, 2017, pp. 381-388, doi: 10.1109/RISE.2017.8378186.
- [4] Martin, Milo & Hill, Mark & Wood, David. (2003). Token coherence. ACM SIGARCH Computer Architecture News. 31. 182. 10.1145/859639.859640.
- [5] R. Fernández-Pascual, J. M. García, M. E. Acacio and J. Duato, "Dealing with Transient Faults in the Interconnection Network of CMPs at the Cache Coherence Level," in IEEE Transactions on Parallel and Distributed Systems, vol. 21, no. 8, pp. 1117-1131, Aug. 2010, doi: 10.1109/TPDS.2009.148.
- [6] B. Fechner, "Analysis of checksum-based execution schemes for pipelined processors," Proceedings 20th IEEE International Parallel & Distributed Processing Symposium, Rhodes, Greece, 2006, pp. 8 pp.-, doi: 10.1109/IPDPS.2006.1639664.
- [7] Dill, D.L. (1996). The Murphi verification system. In: Alur, R., Henzinger, T.A. (eds) Computer Aided Verification. CAV 1996. Lecture Notes in Computer Science, vol 1102. Springer, Berlin, Heidelberg. <https://doi.org/10.1007/3-540-61474-5-86>
- [8] M. M. K. Martin, M. D. Hill and D. A. Wood, "Token Coherence: decoupling performance and correctness," 30th Annual International Symposium on Computer Architecture, 2003. Proceedings., San Diego, CA, USA, 2003, pp. 182-193, doi: 10.1109/ISCA.2003.1206999.
- [9] Grumberg, Orna Veith, Helmut. (2008). 25 Years of Model Checking - History, Achievements, Perspectives. doi: 10.1007/978-3-540-69850-0.
- [10] R. Fernandez-Pascual, J. M. García, M. E. Acacio and J. Duato, "Extending the TokenCMP Cache Coherence Protocol for Low Overhead Fault Tolerance in CMP Architectures," in IEEE Transactions on Parallel and Distributed Systems, vol. 19, no. 8, pp. 1044-1056, Aug. 2008, doi: 10.1109/TPDS.2007.70803.
- [11] R. Fernandez-Pascual, J. M. Garcia, M. E. Acacio and J. Duato, "A Low Overhead Fault Tolerant Coherence Protocol for CMP Architectures," 2007 IEEE 13th International Symposium on High Performance Computer Architecture, Scottsdale, AZ, USA, 2007, pp. 157-168, doi: 10.1109/HPCA.2007.346194.

8. Appendix

8.1. Invariants/Assertions in Murphi

- 1) The value in memory should match the last written value.
- 2) The number of tokens cannot exceed the number of processors in the system by more than one.
- 3) There can be a maximum of one owner token.
- 4) There can be a maximum of one backup token.
- 5) If a processor doesn't have any tokens, its value must be undefined.
- 6) If a message has the owner token, then its value cannot be undefined and the receiving processor cannot already have an owner token.
- 7) The sender of the owner token must have the backup token.
- 8) Main memory can receive a correct SetSerialNumAck only while in the token recreation state.
- 9) If the Set Token Serial Acknowledgement (TrSAck) message contains a valid value, then the memory controller cannot have the owner token.
- 10) If the memory controller sends a token recreation request, it should have the backup token.
- 11) If the memory controller or a processor gets an AckBD, it should be in a blocked ownership state (Mb, Ob).
- 12) Any message containing tokens should have non-zero valid sharer tokens.
- 13) If a processor gets a valid BackupInv message, it cannot have any sharer tokens.
- 14) A processor shouldn't receive TrDone (Token recreation destruction done) while it has a token that isn't the backup token.
- 15) If a TrDone message has a defined value, the recipient processor must be in an invalid state.
- 16) If a processor sends a persistent activation request, it should be added to its persistence table.

8.2. Team Member Contributions

- Advait Iyer: Arjun Laxman: Implemented base data structures to begin formal verification of protocol in Murphi, debugging and verification design in Murphi.
- Arjun Laxman: Advait Iyer: Invariants and rules for formal verification in Murphi. Modifying verification methodology for time based coherence protocol.
- Mustafa Miyaziwala: Designed the state diagram for the coherence protocol from the paper and implemented it in Murphi, debugged the protocol and Murphi and identified deadlocked states.
- Noel Pamenan: Designed the state diagram for the coherence protocol from the paper and implemented it in Murphi, debugged the protocol and Murphi and identified deadlocked states.
- Sanidhya Patel: Implemented base data structures to begin formal verification of protocol in Murphi,

debugged Murphi implementation of the coherence protocol and fixed internal errors.